Triangle mesh. A 3D surface stored as points (vertices) joined into triangles (faces). data/kiri/scene.obj is a triangle mesh in which every vertex also carries a colour, so the model displays with its real scanned appearance.

Surface targets. 60000 individual 3D points spread evenly over the scanned surface, each with an outward normal (see below). They are the measurement grid: coverage is judged at these points, never at mesh vertices. Stored in data/kiri/targets.ply. They lie ON the same surface the mesh represents and share its coordinate frame exactly.

Normal. The direction sticking straight out of the surface at a point, stored as a unit vector. A camera can only usefully observe a target from the side its normal points towards.

Viewpoint (camera viewpoint). One planned UAV camera shot: a position in metres plus an yaw and a pitch. That fully specifies what the camera sees. The plan in data/kiri/plan.json is 185 of these.

Yaw and pitch. Yaw is the compass-like rotation about the vertical axis (measured from the +X axis towards +Y). Pitch tilts the view up (positive) or down (negative). Roll is always zero in this project.

Pose. Position plus orientation packaged as one 4 x 4 matrix, produced by viewscope_core.camera_pose(position, yaw_deg, pitch_deg). You never build this matrix yourself. You ask the library for it.

Frustum. The pyramid-shaped volume a camera can see, with its tip at the camera. Drawing a viewpoint "as a frustum" means drawing that little pyramid so its orientation is visible at a glance.

Rendering. Turning 3D geometry into a 2D image. The library does every photographic render for you (SceneRenderer). What you build in VTK is the interactive viewport, which VTK itself renders.

Offscreen rendering. Rendering into memory instead of a window. The library uses it to produce photographs and depth maps without any GUI.

Depth map. An image where each pixel stores the distance to the nearest surface instead of a colour. It is how occlusion is decided: anything farther than the recorded depth at its pixel is hidden.

Visibility. Whether a camera actually sees a target: the target must be in front of the camera, inside the image, on the side its normal faces, and not hidden behind the building (checked against the depth map).

Occlusion. Being hidden behind geometry. An occluded target is not visible even if the camera points straight at it.

Picking. Clicking in the 3D view and working out which object was clicked. "Pick a surface target" means: the user clicks on the model and your tool selects the nearest surface target to the clicked point.

Sector (directional bin). The compass directions around a target's normal, divided into equal wedges: eight by default, four as the coarse alternative. Every observing camera falls in exactly one wedge (sector_indices). Cameras in different sectors see the target from genuinely different directions, which is what multi-view reconstruction needs.

Weight. A number between 0 and 1 saying how useful one camera is for one target (weight_factors).

Target score. The per-target quality figure: the mean over all sectors of the best weight in that sector, empty sectors counting zero (target_score). This is what the heatmap colours.

Safe placement. The rule refusing a new camera inside the building or closer than the safe distance to the surface (check_placement).

World frame. The single right-handed coordinate system every file shares: +Z up, metres, angles in degrees. Nothing is ever recentred or rescaled, so geometry from any two files can be drawn together directly.

Heatmap. Colouring the surface by target score so weak regions stand out as a colour band rather than a number.

Given versus yours, in one breath. The library computes. Your application decides WHEN to compute, caches what it computed, and shows it. Every formula, constant and geometric test lives in viewscope_core and is imported, never reimplemented. Everything the user sees and touches is yours: the windows, the 3D view, picking, lists, plots, linking, threading and responsiveness.